

מבוא למדעי המחשב

שיעור 1: פייתון בסיסי

לקריאה נוספת:

- J. Fields, S. Harvie, M. Fowler, and K. Beck. *Refactoring: Ruby Edition*. Addison-Wesley, 2013.
- S. McConnell. *Code Complete* (2nd edition). Microsoft Press, 2004.
- D. Knuth. *The Art of Computer Programming* (volumes 1-4). Addison-Wesley, 1968-...
- *Beetlejuice* (1988). [film] Hollywood: Tim Burton (director), The Geffen Film Company (producer), Warner Bros. (distributor).

חזרה

- אובייקטים (objects):
- קבועים מפורשים (literals):

5, -13.27, "abracadabra"

x, y, this_is_a_variable, toB_or_not2B

int, float, str

```
>>> x = "-13.27"
>>> type(x)
<class 'str'>
```

```
>>> y = float(x)
>>> y
-13.27
>>> type(y)
<class 'float'>
>>>
```

```
>>> print("abracadabra", y)      # prints the values of the arguments separated by a space
abracadabra -13.27
>>> x = input("enter a name: ")  # displays the given prompt, returns a string
enter a name:
```

- משתנים (variables):
- טיפוסים (types):
- פקודת השמה (assignment):

- המרות (conversions):

- קלט/פלט:

חזרה (המשך)

- פעולות/אופרטורים (operators):
 - אונריים (unary):

-

**

* / % //

+ -

$2*3*4*5*6$

$2*3 + 4*5$

$2 * (3+4) * 5$

$2*3/4*5*6$

$(2*3) / (4*5*6)$

$2 + 7$

$1.1 + 2.3$

"abra" + "cadabra"

- בינריים (binary):

- ביטויים (expressions):

- קדימות (precedence), אסוציאטיביות (associativity)
- העמסה (overloading):

- לכל ביטוי יש ערך מטיפוס ספציפי

חזרה (המשך)

• הערות (comments):

A comment starts with a hash character (#, a.k.a. number or pound sign) that is not part of a string literal,
and ends at the end of the physical line. Comments are ignored by the interpreter.

#

Any fool can write code that a computer can understand. Good programmers write code that humans
can understand. (Martin Fowler)

#

Good code is its own best documentation. As you're about to add a comment, ask yourself,
“How can I improve the code so that this comment isn't needed?” Improve the code and then
document it to make it even clearer. (Steve McConnell)

#

Computers are good at following instructions, but not at reading your mind. (Donald Knuth)

#

Any code of your own that you haven't looked at for six or more months might as well have been written
by someone else. (Eagelson's Law)

דוגמה

```
*Python 3.4.1: Doppler.py - /Users/ygrabani/Documents/פיית'ון/מבוא למדעי המחשב - הוראה/Code/Doppler.py*

# demonstrate Doppler's equation
# the receiver's position is to the left of the source
# positive source velocity means it's moving away from receiver
# positive receiver velocity means it's moving towards source

emitted_freq = 1000 # emitted frequency of waves in Hz
v_signal = 331.3 # velocity of waves in m/sec (sound in dry air at 0 deg C)
v_source = 0 # velocity of source relative to medium
v_receiver = -100 # velocity of receiver relative to medium

v_at_source = v_signal + v_source # 331.3
v_at_receiver = v_signal + v_receiver # 231.3
ratio = v_at_receiver / v_at_source # 0.6981587684877755
observed_freq = ratio * emitted_freq # 698.1587684877754
print("The observed frequency is: ",observed_freq)

# all at once
observed_freq = (v_signal+v_receiver) / (v_signal+v_source) * emitted_freq
print("The observed frequency is: ",observed_freq)
```

ביצוע מותנה

דוגמה:

יש לכתוב תוכנית על פי המפרט הבא:

קלט: שלושה ערכים a, b, c מטיפוס float

פלט: הדפסה של הפתרונות הממשיים של $ax^2 + bx + c = 0$

אנחנו נשתמש בנוסחה לפיתרון משוואה ריבועית:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

מספר הפתרונות של המשוואה תלוי בסימן של $b^2 - 4ac$ כדי להדפיס כיאות צריך לבדוק את הסימן (חיובי, שלילי, או 0).

ביטויים בוליאניים

- אובייקט בוליאני מקבל אחד משני ערכים: `True` או `False` (אלו שני הקבועים הבוליאניים בפייתון)

- אפשר להרכיב ביטויים שיש להם ערך בוליאני על ידי:
- השוואת ערכים אחרים בעזרת אופרטורים של השוואה דוגמאות:

`b**2 < 4*a*c`

`type(b) is float` # True iff both sides are the same object

רשימת האופרטורים:

`<` `>` `<=` `>=` `==` `!=` `is` `is not`

- הרכבת ביטויים בוליאניים בעזרת אופרטורים בוליאניים דוגמה:

`x >= 0 and x != 5`

האופרטורים הבוליאניים:

`not` `or` `and`

פקודת if

```
if b**2 - 4*a*c < 0:  
    print("We're so sorry, uncle albert,")  
    print("We're so sorry if we caused you any pain.")  
    print("The equation has no real solution :-(")
```

- שימו לב למרכיבים הבאים:
 - מילת המפתח if
 - ביטוי בוליאני
 - נקודותיים
 - בלוק מוזח שמתבצע רק אם ערך הביטוי True

פיצול לשני מקרים

if $b^2 - 4ac < 0$:

print("We're so sorry, uncle albert,")

print("We're so sorry if we caused you any pain.")

print("The equation has no real solution :-(")

else:

print("What do I think of success? It sucks, too much stress")

print("The equation has real solutions!")

print("Regardless where you from")

print("Rise and take a stand")

print("The journey seems extreme")

print("But work towards your dream")

פיצול ליותר מקרים

```
if b**2 - 4*a*c < 0:
    print("Now none is what we have become, we're falling")
    print("Where nobody knows, inside the unknown, still falling")
    print("The equation has no real solution :-(")
elif b**2 - 4*a*c == 0: # קיצור של else if אפשר יותר מבלוק אחד כזה
    print("You're the only one darling,")
    print("There'll never be anyone else but you")
    print("The equation has one real solution :-)")
else: # הבלוק הזה מתבצע אם כל התנאים האחרים לא מתקיימים
    print("Just the two of us")
    print("Building castles in the sky")
    print("Just the two of us")
    print("You and I")
    print("The equation has two real solutions !!!")
```

קינון

Python 3.4.1: quadratic equation.py - /Users/yrabani/Documents/פייתון/מבוא למדעי המחשב/הוראה/Code/quadratic equation.py

```
# solve the quadratic equation  $a*x^2 + b*x + c = 0$ 
# using the closed form solution:  $x = (-b \pm \sqrt{b^2 - 4*a*c}) / (2*a)$ 
import math

a = float(input("enter a: "))
b = float(input("enter b: "))
c = float(input("enter c: "))

if a == 0:
    print("This is a linear equation")
    if b != 0:
        print("The solution is x = ", -c/b)
    elif c == 0:
        print("Every x is a solution")
    else:
        print("The equation has no solution")
else:
    print("This is a true quadratic equation")
    if b**2 < 4*a*c:
        print("The equation has no real solutions")
    elif b**2 == 4*a*c:
        print("The unique real solution is x = ", -b / (2*a))
    else:
        print("There are two solutions:")
        root_disc = math.sqrt(b**2 - 4*a*c)
        print("x1 = ", (-b+root_disc) / (2*a))
        print("x2 = ", (-b-root_disc) / (2*a))
```

לולאות

- לעיתים רוצים לבצע סדרה של פקודות מספר פעמים עם ערכים שונים של המשתנים בכל חזרה.
- דוגמה 1: יש לכתוב תוכנית שקולטת מספרים. כל עוד המספר הבא שנקלט הוא אי-שלילי, יש להדפיס את השורש (האי-שלילי) שלו. אחרת, יש לסיים את ריצת התוכנית.
- דוגמה 2: יש לכתוב תוכנית שקולטת עשרה מספרים ומדפיסה את הממוצע שלהם.

פקודות while, for

Python 3.4.1: repeated_sqrt.py - /Users/yrabani/Documents/פיית'ון/מבוא למדעי המחשב/הוראה/Code/repeated_sqrt.py

```
# read non-negative numbers and print their square roots
# a negative input terminates the program
import math

num = float(input("Enter new number: "))
while num >= 0:
    print("The square root of ", num, " is ", math.sqrt(num))
    num = float(input("Enter new number: "))
```

Ln: 9 Col: 4

Python 3.4.1: average.py - /Users/yrabani/Documents/פיית'ון/מבוא למדעי המחשב/הוראה/Code/average.py

```
# print the average of 10 input numbers

N = 10

sum = 0
for i in range(N):    # range(10) is the sequence 0,1,2,3,4,5,6,7,8,9
    sum += float(input("Enter next number: "))

print("The average input is ", sum / N)
```

Ln: 7 Col: 46

פונקציות

- כימוס (encapsulation):

```
def pretty_reply():  
    print("*****")  
    print("** It's showtime! **")  
    print("*****")
```

- שימוש:

```
print("Beetlejuice, Beetlejuice, Beetlejuice!")  
pretty_reply()
```

- צריך להגדיר פונקציה לפני השימוש בה!

תיעוד

• docstring:

```
>>> def pretty_reply():  
    """ prints the reply of Beetlejuice, being  
        summoned by Lydia """  
    print("*****")  
    print("** It's showtime! **")  
    print("*****")
```

• שימוש:

```
>>> help(pretty_reply)  
Help on function pretty_reply in module __main__:
```

```
pretty_reply()  
  prints the reply of Beetlejuice, being  
  summoned by Lydia
```

```
>>>
```

פרמטרים

```
Python 3.4.1 Shell

Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> def print_twice(message):
    """ prints message twice """
    print(message)
    print(message)

>>> print_twice("double trouble")
double trouble
double trouble
>>> def multiple_print(message,k):
    """ prints message k times, k must be a positive int """
    for i in range(k):
        print(message)

>>> multiple_print("double trouble",2)
double trouble
double trouble
>>> def repeat_and_print(message,k,delim = ' '):
    """ prints a string of k repetitions of message separated by delim
        message must be str, k must be a positive int """
    rep = k * (message+delim)
    print(rep)

>>> repeat_and_print("Beetlejuice",3)
Beetlejuice Beetlejuice Beetlejuice
>>> repeat_and_print("Beetlejuice",3,", ")
Beetlejuice, Beetlejuice, Beetlejuice,
>>>
```

תחום ההגדרה של משתנים (scope)

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> x = 1
>>> def demo1():
    y = 2
    print(x,y)

>>> demo1()
1 2
>>> y
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    y
NameError: name 'y' is not defined
>>> def demo2():
    x = 3
    y = 2
    print(x,y)

>>> demo2()
3 2
>>> x
1
>>> y = 2
>>> def demo3(x):
    y = 4
    print(x,y)

>>> demo3(x)
1 4
>>> demo3(y)
2 4
>>>
```

החזרת ערכים

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> def my_divide(x,y):
    """ returns x // y without using the operator //
        x,y must be non-negative int, y != 0 """
    result = 0
    remainder = x
    while remainder >= y:
        result += 1      # same as result = result + 1
        remainder -= y  # same as remainder = remainder - y
    return result

>>> my_divide(11,2)
5
>>> def my_divmod(x,y):
    """ returns x // y and x % y
        x, y must be int """
    return x//y, x%y

>>> my_divmod(11,2)
(5, 1)
>>> a, b = my_divmod(11,2)
>>> print(a,b)
5 1
>>> def divisor(x,y):
    """ returns True if y divides x, False otherwise
        both x, y must be int """
    if y * my_divide(x,y) == x:
        return True
    else:
        return False

>>> divisor(11,2)
False
>>>
```

החזרת ערכים

```
Python 3.4.1 Shell
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> def my_divide(x,y):
    """ returns x // y without using the operator //
        x,y must be non-negative int, y != 0 """
    result = 0
    remainder = x
    while remainder >= y:
        result += 1      # same as result = result + 1
        remainder -= y  # same as remainder = remainder - y
    return result

>>> my_divide(11,2)
5
>>> def my_divmod(x,y):
    """ returns x // y and x % y
        x, y must be int """
    return x//y, x%y

>>> my_divmod(11,2)
(5, 1)
>>> a, b = my_divmod(11,2)
>>> print(a,b)
5 1
>>> def divisor(x,y):
    """ returns True if y divides x, False otherwise
        both x, y must be int """
    if y * my_divide(x,y) == x:
        return True
    else:
        return False

>>> divisor(11,2)
False
>>>
```

זה לא קוד טוב

עוד דוגמה

Python 3.4.1 Shell

```
Python 3.4.1 (v3.4.1:c0e311e010fc, May 18 2014, 00:54:21)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> def convert_date(date):
    """ converts dd/mm/yyyy to day, month, year
        the input must be a 10 character string
        the output is 3 ints """
    day = int(date[:2])
    month = int(date[3:5])
    year = int(date[6:])
    return day, month, year

>>> convert_date("06/03/2017")
(6, 3, 2017)
>>> day, month, year = convert_date("01/10/1001")
>>> print(day, month, year)
1 10 1001
>>>
```

תכנות מודולרי

- עיקרון המודולריות (modularity) - תוכנית מורכבת מחלקים ליחידות נבדלות שניתן להחליף בלי לשנות יחידות אחרות.
- פונקציות הן כלי למימוש העיקרון הזה.
- כל פונקציה צריכה להיות ממוקדת, קצרה ופשוטה.
- לכל פונקציה צריך להיות ממשק מוגדר וברור.
- די בדיעת הממשק כדי להשתמש בפונקציה.
- השפעה בין פונקציות מתרחשת רק דרך הממשק שלהן.

הממשק של פונקציה

- תיאור הפעולה שהפונקציה מבצעת.
- המשמעות של הפרמטרים והתנאים שהפונקציה מצפה מהם לקיים בעת הפעלתה (טיפוס, תחום ערכים).
- המשמעות של הערכים שמחזירה הפונקציה והתנאים שהערכים הללו מקיימים.
- מקובל לתעד את הממשק ב-docstring, כלומר מחרוזת התחומה בשלושה זוגות גרשיים בתחילתה ובסופה.
- מחרוזת כזו יכולה להשתרע על פני מספר שורות.
- שמים את התיעוד מייד אחרי חתימת הפונקציה.
- הפעלת `help(func)` מדפיסה את ה-docstring שמופיע בתוך הפונקציה `func`.
- לחילופין אפשר לבצע את הפקודה `print(func.__doc__)`.
- יש להתייחס לתיעוד כאל חוזה שמפרט מה הפונקציה מבטיחה לבצע ומה התנאים שהיא דורשת לשם כך.

The Zen of Python

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

Tim Peters

<https://www.python.org/dev/peps/pep-0020/#id3>